PracticeAssessment_SMS_SpamCollection_using_the_NaïveBayes_spam_fi

November 2, 2026

1 Hands-on Lab: SMS Spam Collection using the Naïve Bayes spam filter

Estimated Time: **40** Minutes

1.0.1 Overview:

In this lab, you'll develop a spam filter tool that processes email or SMS text bodies from a text file to determine whether it's spam or ham using Naïve Bayes Algorithm.

1.0.2 Learning Objectives:

- Learn how the Naïve Bayes algorithm works and why it's effective for text classification tasks like spam filtering.
- Gain hands-on experience in preprocessing text data, including tokenization, stop-word removal, and feature extraction, to prepare it for machine learning models.
- Learn how to assess the accuracy and effectiveness of your spam filter by testing it with unseen data and analyzing its performance metrics.
- Finally, develop a spam filter using Python, leveraging the Naïve Bayes algorithm to classify emails or SMS messages as spam or ham.

1.1 Design Features

To accomplish this, utilize the dataset I provided along with the ML analytic development process to create a spam filter that dynamically identifies its own suspect keywords. After developing the model, export it and write code to use the exported model for performing the spam filtering as described.

1.2 Problem Statement:

Design and implement a spam filter tool inspired by the Naïve Bayes spam filter. The tool should take an email or SMS text body as input in the form of a text file and classify it as either spam or ham. The model should dynamically recognize suspect keywords using a provided dataset and follow the machine learning analytic development process.

1.2.1 Requirements

- Python 3.x

- scikit-learn
- pandas
- nltk

1.2.2 Step 1: Import Required Packages

[13]:
```
import pandas as pd

# Load the dataset
data = pd.read_csv('SMSSpamCollection.csv')  # Replace with your dataset path
data.head()
```

[13]:

	Label	Message
0	ham	Go until jurong point, crazy.. Available only …
1	ham	Ok lar… Joking wif u oni…
2	spam	Free entry in 2 a wkly comp to win FA Cup fina…
3	ham	U dun say so early hor… U c already then say…
4	ham	Nah I don't think he goes to usf, he lives aro…

1.2.3 Step 2: Preprocess The Data

- The labels ("spam" or "ham") are converted into binary values (1 for spam, 0 for ham).
- The text data is extracted and prepared for vectorization.
- Text Cleaning
- Feature Extraction
- TF-IDF (Term Frequency-Inverse Document Frequency)

Explanation Text Cleaning:

- The preprocess_text function cleans each message by converting it to lowercase, removing non-alphabetic characters, tokenizing it, and removing stop words. The cleaned text is stored in the cleaned_message column.

Feature Extraction: - Bag of Words (BoW): The CountVectorizer converts the cleaned messages into a matrix of token counts. - TF-IDF: The TfidfVectorizer transforms the cleaned messages into a TF-IDF matrix.

[14]:
```
# Write your code here!
#Step 1:
# Convert the labels into binary values (1 for spam, 0 for ham)
# Convert labels: spam = 1, ham = 0
data['Label'] = data['Label'].map({'spam': 1, 'ham': 0})

data.head()
```

[14]:

	Label	Message
0	0	Go until jurong point, crazy.. Available only …
1	0	Ok lar… Joking wif u oni…
2	1	Free entry in 2 a wkly comp to win FA Cup fina…

```
3      0  U dun say so early hor… U c already then say…
4      0  Nah I don't think he goes to usf, he lives aro…
```

Click here to view/hide solution

```
data['Label'] = data['Label'].map({'spam': 1, 'ham': 0})
```

[15]:
```
#Step 2: Text Cleaning:
#Import required packages
import re
import nltk
import pandas as pd
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /home/jovyan/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

[15]:
```
True
```

[16]:
```
# Write your code here!
from nltk.tokenize import word_tokenize

#Step 3:
nltk.download('punkt')
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation, special characters, and numbers
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize the text (split into words)
    tokens = text.split()
    # Remove stop words
    tokens = [word for word in tokens if word not in stop_words]
    # Join the tokens back into a single string
    return ' '.join(tokens)

# Apply preprocessing to the 'message' column
data['cleaned_message'] = data['Message'].apply(preprocess_text)

# Display the first few rows of the cleaned data
data[['Message', 'cleaned_message']].head()
```

```
[nltk_data] Downloading package punkt to /home/jovyan/nltk_data…
[nltk_data]   Package punkt is already up-to-date!
```

[16]:
```
                                                Message  \
0  Go until jurong point, crazy.. Available only …
1                      Ok lar… Joking wif u oni…
2  Free entry in 2 a wkly comp to win FA Cup fina…
3  U dun say so early hor… U c already then say…
4  Nah I don't think he goes to usf, he lives aro…

                                    cleaned_message
0  go jurong point crazy available bugis n great …
1                            ok lar joking wif u oni
2  free entry wkly comp win fa cup final tkts st …
3                u dun say early hor u c already say
4        nah dont think goes usf lives around though
```

Click here to view/hide solution

```
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation, special characters, and numbers
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize the text (split into words)
    tokens = text.split()
    # Remove stop words
    tokens = [word for word in tokens if word not in stop_words]
    # Join the tokens back into a single string
    return ' '.join(tokens)

# Apply preprocessing to the 'message' column
data['cleaned_message'] = data['Message'].apply(preprocess_text)

# Display the first few rows of the cleaned data
print(data[['Message', 'cleaned_message']].head())
```

[19]:
```
# Write your code here!
# Step 4: Feature Extraction
# Part 1 :Bag of Words (BoW)
vectorizer = CountVectorizer()
# Transform the cleaned text data into a matrix of token counts
X_bow = vectorizer.fit_transform(data['cleaned_message'])

# Convert the BoW matrix to a DataFrame for better readability (optional)
bow_df = pd.DataFrame(X_bow.toarray(), columns=vectorizer.
 ↪get_feature_names_out())

# Display the first few rows of the BoW DataFrame
```

```
print("Bag of Words (BoW):")
bow_df.head()
```

```
Bag of Words (BoW):
```

[19]:

	aa	aah	aaniye	aaoooright	aathilove	aathiwhere	ab	abbey	abdomen	\
0	0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	

	abeg	…	zeros	zf	zhong	zindgi	zoe	zogtorius	zoom	zouk	zs	zyada
0	0	…	0	0	0	0	0	0	0	0	0	0
1	0	…	0	0	0	0	0	0	0	0	0	0
2	0	…	0	0	0	0	0	0	0	0	0	0
3	0	…	0	0	0	0	0	0	0	0	0	0
4	0	…	0	0	0	0	0	0	0	0	0	0

```
[5 rows x 8480 columns]
```

Click here to view/hide solution

```
vectorizer = CountVectorizer()
# Transform the cleaned text data into a matrix of token counts
X_bow = vectorizer.fit_transform(data['cleaned_message'])

# Convert the BoW matrix to a DataFrame for better readability (optional)
bow_df = pd.DataFrame(X_bow.toarray(), columns=vectorizer.get_feature_names_out())

# Display the first few rows of the BoW DataFrame
print("Bag of Words (BoW):")
print(bow_df.head())
```

[20]:

```
# Step 4: Feature Extraction
tfidf_vectorizer = TfidfVectorizer()
# Transform the cleaned text data into a TF-IDF matrix
X_tfidf = tfidf_vectorizer.fit_transform(data['cleaned_message'])

# Convert the TF-IDF matrix to a DataFrame for better readability (optional)
tfidf_df = pd.DataFrame(X_tfidf.toarray(), columns=tfidf_vectorizer.
 ↪get_feature_names_out())

# Display the first few rows of the TF-IDF DataFrame
print("\nTF-IDF:")
tfidf_df.head()
```

TF-IDF:

[20]:
```
    aa  aah  aaniye  aaoooright  aathilove  aathiwhere   ab  abbey  abdomen  \
0  0.0  0.0     0.0         0.0        0.0         0.0  0.0    0.0      0.0
1  0.0  0.0     0.0         0.0        0.0         0.0  0.0    0.0      0.0
2  0.0  0.0     0.0         0.0        0.0         0.0  0.0    0.0      0.0
3  0.0  0.0     0.0         0.0        0.0         0.0  0.0    0.0      0.0
4  0.0  0.0     0.0         0.0        0.0         0.0  0.0    0.0      0.0

   abeg  …  zeros   zf  zhong  zindgi  zoe  zogtorius  zoom  zouk   zs  \
0   0.0  …    0.0  0.0    0.0     0.0  0.0        0.0   0.0   0.0  0.0
1   0.0  …    0.0  0.0    0.0     0.0  0.0        0.0   0.0   0.0  0.0
2   0.0  …    0.0  0.0    0.0     0.0  0.0        0.0   0.0   0.0  0.0
3   0.0  …    0.0  0.0    0.0     0.0  0.0        0.0   0.0   0.0  0.0
4   0.0  …    0.0  0.0    0.0     0.0  0.0        0.0   0.0   0.0  0.0

   zyada
0    0.0
1    0.0
2    0.0
3    0.0
4    0.0

[5 rows x 8480 columns]
```

Click here to view/hide solution

```
tfidf_vectorizer = TfidfVectorizer()
# Transform the cleaned text data into a TF-IDF matrix
X_tfidf = tfidf_vectorizer.fit_transform(data['cleaned_message'])

# Convert the TF-IDF matrix to a DataFrame for better readability (optional)
tfidf_df = pd.DataFrame(X_tfidf.toarray(), columns=tfidf_vectorizer.get_feature_names_out())

# Display the first few rows of the TF-IDF DataFrame
print("\nTF-IDF:")
print(tfidf_df.head())
```

1.2.4 Step 3: Train Naive Bayes Model

- The data is split into training and testing sets.
- A MultinomialNB Naive Bayes classifier is trained on the training data.

[21]:
```
#Step 1: Import required packages.
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix,␣
 ↪classification_report
```

[22]:
```
# Write your code here!
# Define the feature matrix and target vector
X = data['cleaned_message']  # Feature matrix (cleaned messages)
y = data['Label']  # Target vector (labels: 'spam' or 'ham')
# For Bag of Words:
X_features = vectorizer.fit_transform(X)

# For TF-IDF (uncomment the following line if you prefer TF-IDF):
# X_features = tfidf_vectorizer.fit_transform(X)

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_features, y, test_size=0.
 ↪2, random_state=42)

# Display the size of each set
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")

# Write your code here!
```

```
Training set size: 4459 samples
Testing set size: 1115 samples
```

Click here to view/hide solution

```
# For Bag of Words:
X_features = vectorizer.fit_transform(X)

# For TF-IDF (uncomment the following line if you prefer TF-IDF):
# X_features = tfidf_vectorizer.fit_transform(X)

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X_features, y, test_size=0.2, random_state=

# Display the size of each set
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")
```

[23]:
```
# Write your code here!
#Step 2: Train the model
# Initialize the Multinomial Naive Bayes model
model = MultinomialNB()

# Train the model using the training data

model.fit(X_train, y_train)
```

[23]:
```
MultinomialNB()
```

Click here to view/hide solution

```
model = MultinomialNB()

# Train the model using the training data
model.fit(X_train, y_train)
```

1.2.5 Step 4: Evaluate The Model

- The model is tested on the testing set, and metrics like accuracy, precision, recall, and F1-score are printed.

[24]:
```
# Write your code here!
# Evaluate the model

# Predict the labels for the test set
y_pred = model.predict(X_test)

# Display the predicted labels
print("Predicted labels:")
print(y_pred)
```

```
Predicted labels:
[0 0 0 … 0 0 1]
```

Click here to view/hide solution

```
# Predict the labels for the test set
y_pred = model.predict(X_test)

# Display the predicted labels
print("Predicted labels:")
print(y_pred)
```

[25]:
```
# Write your code here!

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Generate and print the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)

# Generate and print the classification report
class_report = classification_report(y_test, y_pred, target_names=['ham',␣
↪'spam'])
print("Classification Report:")
```

```
print(class_report)
```

```
Accuracy: 0.97
Confusion Matrix:
[[927  27]
 [ 10 151]]
Classification Report:
              precision    recall  f1-score   support

         ham       0.99      0.97      0.98       954
        spam       0.85      0.94      0.89       161

    accuracy                           0.97      1115
   macro avg       0.92      0.95      0.94      1115
weighted avg       0.97      0.97      0.97      1115
```

Click here to view/hide solution

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Generate and print the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)

# Generate and print the classification report
class_report = classification_report(y_test, y_pred, target_names=['ham', 'spam'])
print("Classification Report:")
print(class_report)
```

1.2.6 Step 5: Verify The Model

- A function classify_message() is provided to classify new SMS messages as "spam" or "ham."

[26]:
```
def classify_message(message):
    vec_message = vectorizer.transform([message])
    prediction = model.predict(vec_message)
    return "Spam" if prediction == 1 else "Ham"

# Example usage
new_message = "Thanks for your subscription to Ringtone UK your mobile will be␣
 ↪charged £5/month Please confirm by replying YES or NO. If you reply NO you␣
 ↪will not be charged"
print(f"Message: {new_message}\nClassification:␣
 ↪{classify_message(new_message)}")
```

```
Message: Thanks for your subscription to Ringtone UK your mobile will be charged
£5/month Please confirm by replying YES or NO. If you reply NO you will not be
charged
Classification: Spam
```

Click here to view/hide solution

```
def classify_message(message):
    vec_message = vectorizer.transform([message])
    prediction = model.predict(vec_message)
    return "Spam" if prediction == 1 else "Ham"

# Example usage
new_message = "Thanks for your subscription to Ringtone UK your mobile will be charged £5/montl
print(f"Message: {new_message}\nClassification: {classify_message(new_message)}")
```

1.2.7 Summary:

A spam filter tool was developed using a Naïve Bayes model to classify email or SMS text as either spam or ham. The tool takes a message as input, transforms it using a vectorizer, and predicts its classification based on the trained model. In the example code provided, a sample message about a ringtone subscription is classified as either spam or ham. The tool dynamically identifies suspicious keywords and leverages a trained model to make predictions, offering a user-friendly interface to detect spam.

In addition to individual message classification, the tool can be used to verify and classify multiple messages by utilizing statements from a provided CSV file. Each message in the CSV is processed, transformed, and classified as either spam or ham using the trained Naïve Bayes model. This allows for batch processing of messages, making the tool efficient for analyzing large datasets while dynamically identifying key patterns indicative of spam.